

PROGRAMMING II

JAVA CLASSES AND OBJECTS

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

JAVA CLASS



PROGRAMMING IN JAVA

- ▶ In Java, everything is an object
- ▶ An object is a value of a specific type, defined by a **class**

To program in Java, you need to define classes!

```
public class Car { // class name
    // Class fields -- The variables each object will contain
    String color;
    boolean turnedOn;

    // Class methods -- The messages each objects can accept
    void paint(String newColor) {
        color = newColor; // for instance, set to 'red'
    }
    boolean isTurnedOn() {
        return turnedOn;
    }
}
```

JAVA METHOD

The messages we can send to an objects have a **signature**

returnType methodName(parameter₁ , ... , parameter_n)

where parameters are typed variable names: *parameterType parameterName*

For instance, the method `String getCarMainColor()`

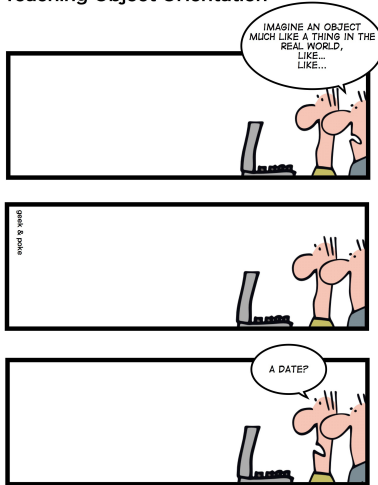
- ▶ Has no parameters, but it returns an integer

While, the method `void paint(String mainColor, String topColor)`

- ▶ Has two parameters, but no return value (the keyword `void` is needed)

CLASS-IC EXAMPLE

Teaching Object Orientation



JAVA EXECUTION

Nice, but

- ▶ How can we run a class?
- ▶ How can we send messages to an object?

A class can be run only if it contains a special method called `main`

- ▶ In that case, the run is automatically performed by the JVM

To send a message to an object, one have to

- ▶ First create the object in memory (from another object)
- ▶ Then call one of its methods

JAVA CLASS ENTRYPOINT

```
public class HelloWorld {                                1
                                                         2
    // Class entrypoint -- This method is called to start execution  3
    public static void main(String[] args) {              4
        System.out.println("Hello_World!");              5
    }                                                       6
                                                         7
}                                                           8
```

The parameter `args` is a list of strings representing command-line arguments

- ▶ No class/executable name is passed via `args`

The entrypoint must be `static`, meaning that only one entrypoint does exist

JAVA STATIC METHODS

The keyword `static` makes a method attached to the class rather than its objects

A static method

- ▶ Can be called even if no objects of the class have been created
- ▶ Cannot access object-specific content (e.g., non-static fields)
- ▶ Can only be called via class name: `HelloWorld.main()`

Static methods are useful to perform actions not related to any class instance

- ▶ Utility or helper classes
- ▶ Side-effects free computations

JAVA OBJECTS



CLASS INSTANCE

To actually use an object one should allocate some memory and create a reference

This is done by using the **new** keyword

- ▶ From an already instantiated object: `Car myCar = new Car();`
- ▶ The JVM automatically reserves the memory for a Car object
- ▶ The JVM provides a reference to the new object into the variable `myCar`

Recall that a reference is a class name and a memory location (e.g., `Car@36c7a21b`)

CLASS INSTANCE

Zero, one or more references can point to the same object

```
Car car1, car2; 1
2
car1 = new Car(); // the Car@36c7a21b object is created 3
car1.paint("yellow") // set color to 'yellow' 4
// one reference to the yellow car 5
6
car2 = car1; 7
// two references to the yellow car 8
car2 = null; 9
// one reference to the yellow car 10
car1 = null; 11
// no reference to the yellow car again 12
13
// the Car@36c7a21b object is freed by the garbage collector 14
```

OBJECT CREATION

The instruction `var = new className()`

- ▶ Allocates the memory in the **heap** for a `className` object
- ▶ Creates a new **instance** of the class `className`
- ▶ Calls the **constructor** of the class `className`
- ▶ Returns a reference to the new `className` object into `var`

// The constructor is a special method used to initialize class objects

INTERACTING WITH AN OBJECT

Once instantiated, we send a message to an object by calling one of its methods

- ▶ The method to call must be not **static**

```
public class MainCar {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.paint("yellow") // set color to 'yellow'
        System.out.println("Yellow car created!");
    }
}
```

JAVA INSTANCE METHODS

Without the keyword `static` a method will be attached to class instances (objects)

An instance method

- ▶ Can be called only from instantiated objects
- ▶ Can access object-specific and class content (e.g., non-static and static fields)
- ▶ Can only be called via object reference: `myCar.paint("yellow")`

✎ From a static context one cannot call a non-static method, while from a non-static context one can call a static method

JAVA STATIC VS INSTANCE METHODS

```
public class MainCar {                                     1
                                                            2
    public static void main(String[] args) {              3
        Car myCar = new Car();                             4
        myCar.paint("yellow") // instance method          5
        System.out.println("Yellow car created!"); // static method 6
    }                                                       7
                                                            8
}                                                           9
```

The static method `println()` of the static field out of the class `System` is called

- ▶ `System` is a standard library provided by the JVM

JAVA STATIC VS INSTANCE METHODS (CONT'D)

```
public class Car {
    String color;

    public void paint(String newColor) {
        color = newColor;
        this.printColor(); // The current object is retrieved -- The
    }                      // compiler implicitly adds this. if omitted

    public void printColor() {
        System.out.println(color);
    }
}
```

CLASS FIELDS

Also fields can be accessed by using the dot-notation

<code>Car myCar = new Car();</code>	1
<code>String carColor = myCar.color; // car color is 'yellow'</code>	2
<code>myCar.color = "red"; // change car color to 'red'</code>	3

This is actually bad, as it violates the [information hiding](#) principle

CLASS FIELDS (CONT'D)

Again, inside an instance method we can refer the current object with **this**

```
public class Car {                                1
    String color;                                  2

    public void paint(String newColor) {           3
        this.color = newColor; // The current object is retrieved -- The 4
    } // compiler implicitly adds this. if omitted 5
}                                                  6
                                                  7
```

MANDATORY CURRENT OBJECT REFERENCE

In some cases, the keyword `this` is mandatory

- To distinguish class fields from method parameters

```
public class Car {                                1
    String color;                                  2

    public void paint(String color) {              3
        this.color = color; // Ok -- It is now clear that the class 4
    } // field is assigned with the method parameter 5
}                                                  6
                                                  7
```

REFERENCE TYPE



REFERENCES

Each class `C` yields a reference type T_C parametric on that class

- ▶ Every value of T_C is a reference to `C` instances
- ▶ That is, a pointer to a memory area containing a `C` object

Except for primitive values, variables in Java can only hold references

- ▶ The special value `null` belongs to every reference type
- ▶ Such a value can be assigned to all and only variables holding references

```
Clazz clazz = null; // Ok
String str = null; // Ok
int i = null; // Compilation error
```

1
2
3

REFERENCES (CONT'D)

When a reference holds the value `null`, it does not point to any memory area

The dot-notation automatically performs pointer `dereference`

- ▶ `str.toString()` will access the memory pointed by `str`
- ▶ Dereferencing `null` thus yields an exception

Setting a reference to `null` may wake up the garbage collector

PRINT A REFERENCE

Every object in Java comes with the `toString()` method *// we will see later why*

- ▶ This method provides a string representation of an object
- ▶ It represents a good approximation of how a reference looks like

```
Car myCar = new Car();  
System.out.println(myCar.toString());  
// the output will be something like 'Car@5acf9800'
```

1
2
3

PRINT A REFERENCE (CONT'D)

The method `toString()` is implicitly called in contexts where a string is expected

```
String str = "Reference_=_ " + myCar;           1
System.out.println(str); // also here toString() is implicitly called 2
// the output will be something like 'Reference = Car@5acf9800'        3
```

Why does `str.toString()` not output something like `String@4617c264` ?

💡 With **overriding** one can change the implementation of a method

Q + A

